

WoodStickProjectVer1.0

엔진 아키텍처 및 데이터 플로우 사양서

문서 버전: 1.0 / 작성일: 2026-06-13 / 범위: Java 엔진, Data 에셋, Lua 콘텐츠 로직, 런타임 메서드 목록

요약. 이 프로젝트는 Java가 렌더링, 입력, 물리, 오디오, Lua 브리지를 담당하고, Data 폴더의 Lua와 에셋이 실제 콘텐츠 제작 자유도를 담당하는 데이터 드리븐 구조다. 현재 게임은 랜덤 미로에서 아이템과 무기를 활용해 적을 처치하거나 회피하면서 출구에 도달하는 2.5D 미로 탈출 액션 게임이다.

1. 문서 목적과 범위

이 문서는 팀원이 프로젝트를 빠르게 이어받을 수 있도록 엔진 전체 아키텍처, 데이터 흐름, 런타임 책임 분리, 사용 라이브러리, 프로젝트 정의 메서드와 Lua 함수 목록을 정리한다.

- 범위: C:/new/WoodStickProjectVer1.0 기준 src/engine Java 코드, Data/Data.json, Data/Script Lua 코드, Data/Level 맵/씬 파일.
- 메서드 범위: 프로젝트에서 직접 정의한 Java 메서드, Lua에 노출된 engine API, Data/Script에 정의된 Lua 함수.
- 외부 라이브러리 범위: lib 폴더 JAR, Java 표준 API, Lua 스크립트에서 사용한 Lua 표준 기능.

2. 시스템 개요

영역	주 책임	대표 파일
Java Core	윈도우, 메인 루프, 입력 스냅샷, 물리, 렌더, 오디오, Lua 호출 순서를 조립한다.	src/engine/Core.java
Boot/Registry	Data 폴더를 스캔해 asset id/index/path 레지스트리를 만들고 텍스처 초기화를 수행한다.	boot/AssetRegistry*.java, Boot.java
Render	Wolfenstein식 DDA 레이캐스팅으로 벽/바닥/천장을 그리고 스프라이트와 UI를 합성한다.	render/RenderCore.java, SpriteRenderer.java, Texture.java, UiManager.java
Physics/Input	플레이어 이동과 동적 엔티티 벽 충돌을 처리한다.	Control.java, PhysicsCore.java, PlayerController.java
Script Bridge	Java 서비스를 Lua의 engine 테이블에 함수로 노출하고 엔티티 update를 호출한다.	script/ScriptAPI.java, ScriptManager.java
Data Logic	타이틀, 로딩, 미로 생성, 스폰, 몬스터 AI, 전투, UI HUD, 아이템 획득 로직을 담당한다.	Data/Script/*.lua
Assets/Levels	이미지, 오디오, map.dat, scene.json, Lua 파일을 asset registry 대상으로 제공한다.	Data/**

3. 레이어별 아키텍처

엔진은 "Java 런타임 처리기 + Lua 콘텐츠 레이어"에 가깝다. Java 쪽은 기능 단위가 단순하고 public 필드 기반 컴포넌트를 Lua에 노출한다. Lua 쪽은 콘텐츠 제작자가 씬 전환, 오브젝트 스폰, UI draw

command, 전투 판정을 직접 구성하는 구조다.

레이어	상위에서 보는 역할	하위 의존
Application	Core가 전체 라이프사이클과 프레임 순서를 결정한다.	Boot, AssetRegistry, ScriptManager, RenderCore, PhysicsCore
Runtime Services	텍스처/사운드/UI/입력/씬을 보관하거나 처리한다.	Texture, SoundEngine, UiManager, Control, Scene
Script API	Lua에서 엔진 서비스를 호출할 수 있게 engine.* 함수를 등록한다.	Boot, Texture, UiManager, SoundEngine, AssetRegistry
Content Scripts	실제 게임 상태와 콘텐츠 규칙을 구현한다.	Data/Script/*.lua, engine API
Asset Data	이미지, WAV, map.dat, scene.json, Lua 파일을 스캔 대상 자원으로 제공한다.	Data/Data.json

4. 부팅 및 실행 순서

- 1 Core 생성자가 Boot를 만들고 Config 기본값을 로드한다. 현재 src/engine/config.json은 파싱되지 않는다.
- 2 AssetRegistryBuilder가 Data 폴더를 스캔해 Data/Data.json을 재생성한다.
- 3 AssetRegistry가 Data/Data.json을 읽어 id, index, path 맵을 구성한다.
- 4 Texture, UiManager, SoundEngine이 생성되고 Textures.UI.* 에셋은 UiManager에 별도 등록된다.
- 5 Boot.init(Texture)가 Textures 카테고리 에셋을 Texture에 로드한다.
- 6 ScriptManager가 Lua globals를 생성하고 ScriptAPI.register로 engine 테이블을 등록한다.
- 7 Script.main이 실행되어 _G.GameState, _G.PlayerState를 만들고 title.lua를 로드한다.
- 8 Control, PhysicsCore, PlayerController, RenderCore를 만든 뒤 JFrame/Canvas를 띄운다.
- 9 start()가 engine_Main_Loop 스레드를 시작하고 run()이 update/render 루프를 돈다.

5. 프레임 업데이트 데이터 플로우

프레임 갱신의 중심은 Core.update(deltaTime)이다. Java는 순서를 보장하고, 실질적인 게임 규칙은 대부분 Lua update에서 실행된다.

순서	처리	데이터 변화
1	Control.snapshot()	키 입력 volatile 필드를 s_* 프레임 상태로 복사한다.
2	Scene 조회	Boot.currentScene이 없으면 프레임 처리를 중단한다.
3	PlayerController.update()	WASD/방향키 기반으로 player.physics.velX/Y와 camera 방향을 갱신한다.
4	ScriptManager.updateEntity()	플레이어 외 엔티티의 scriptPath에 연결된 Lua update(entity,dt,player,control)를 호출한다.
5	Sound request 처리	entity.sound.requestPlay가 있으면 AssetRegistry 경로를 찾아 SoundEngine.playSfx를 호출한다.
6	PhysicsCore.update()	동적 엔티티를 이동시키고 벽 충돌을 축별로 처리한다.
7	RenderCore.render()	바닥/천장, 벽, 스프라이트, UI draw command를 프레임버퍼에 그린다.

6. 에셋 레지스트리 데이터 플로우

Data 폴더는 런타임 데이터베이스처럼 사용된다. 시작 시 매번 스캔되므로 파일 추가/삭제가 Data/Data.json에 반영된다.

단계	입력	출력/효과
Scan	Data 아래 png, wav, dat, json, lua	카테고리별 AssetInfo(index,id,path) 목록
ID 생성	Data 기준 상대 경로	예: Data/Textures/Level/Wall_1.png -> Textures.Level.Wall_1
Index 부여	스캔 순서	0x00~0xFF 범위. 초과 시 빌드 오류 로그
JSON 생성	categorizedAssets	Data/Data.json
Runtime load	Data/Data.json	AssetRegistry.paths/indexes/indexPaths/indexIds
Texture/UI bind	AssetRegistry ids	Texture wall/flat 캐시, UiManager 이미지 캐시

7. 씬 및 미로 생성 데이터 플로우

현재 scene.json은 선언적 씬의 틀만 있고, 실제 로딩은 Lua가 engine.initScene(sceneName,mapId)를 호출해 map.dat를 Java로 읽히는 방식이다.

- 1 title.lua가 타이틀 map.dat를 로드하고 Title_Controller 엔티티를 Script.title로 스폰한다.
- 2 Start 선택 시 loading.lua가 MazeGenerator.new(81,81,os.time())로 미로 생성을 시작한다.
- 3 loading.lua update가 animated DFS를 진행하며 생성 진행률과 미리보기 UI를 그린다.
- 4 생성 완료 시 ObjectSpawner가 시작점, 출구, 적, 아이템, 무기를 _G.GameState.mazeObjects에 배치한다.
- 5 MazeGenerator.saveToFile("Data/Level/maze/")가 생성 맵을 Data/Level/maze/map.dat에 저장한다.
- 6 maze.lua가 engine.initScene("Maze","Level.maze.map")으로 방금 저장된 map.dat를 Java Scene으로 로드한다.
- 7 _G.GameState.mazeObjects를 순회해 ObjectSpawn.spawn이 각 오브젝트를 엔티티와 Lua 스크립트로 연결한다.
- 8 플레이 중 출구 거리 1.0 이하에서 Enter를 누르면 escape.lua로 전환한다. 체력이 0 이하이면 end.lua로 전환한다.

8. 데이터 형식과 런타임 계약

데이터	형식	런타임 계약
Data/Data.json	카테고리 -> 배열[{index,id,path}]	AssetRegistry의 단일 소스. id는 Lua/Java에서 asset lookup 키로 사용된다.
map.dat	공백 구분 정수 그리드	0은 길, 1 이상은 벽/타일 코드. Java initScene이 int[width][height]로 변환한다.
scene.json	name/map/entities 구조	현재 직접 로딩되지 않는다. 향후 선언형 씬 로더 확장 포인트다.
_G.GameState	Lua 전역 테이블	currentScene, maze, start, exit, mazeObjects, debugNoDeath 등을 보관한다.
_G.PlayerState	Lua 전역 테이블	hp, weapon, ammo, damageFlashTimer, crosshairHitTimer 등 게임 상태를 보관한다.

데이터	형식	런타임 계약
Entity	Java public 필드 + 컴포넌트	Lua userdata로 노출되어 entity.physics.x 같은 직접 접근이 가능하다.

9. 렌더링 파이프라인

렌더링은 2.5D 레이캐스팅 구조다. RenderCore는 고정 크기 BufferedImage를 프레임버퍼로 보유하고 매 프레임 int[] pixels를 갱신한다.

- 바닥/천장: 플레이어 카메라 방향과 plane 벡터로 행 단위 월드 좌표를 계산하고 반복 UV 텍스처를 샘플링한다.
- 벽: 화면 x 컬럼마다 DDA로 벽 타일을 찾고, Texture.getWallPixel(tileId,u,v)로 세로 라인을 그린다.
- zBuffer: 각 컬럼의 벽 깊이를 저장해 SpriteRenderer가 벽 뒤 스프라이트를 제외한다.
- 스프라이트: 활성 엔티티를 거리 역순으로 정렬하고 카메라 공간으로 투영해 이미지 픽셀을 합성한다.
- UI: Entity.ui 오버레이와 UiManager draw command를 최종 프레임버퍼에 합성한다.

10. 사용 라이브러리 및 API

라이브러리/API	사용 위치	역할
luaj-jse-3.0.2.jar	lib, ScriptManager, ScriptAPI, Entity/Control wrapper	Java에서 Lua 스크립트를 로드/실행하고 Java 객체를 Lua userdata로 노출한다.
json-20240303.jar / org.json	lib, AssetRegistry	Data/Data.json 파싱에 사용한다.
Java Swing/AWT	Core, Control	JFrame/Canvas 윈도우, BufferStrategy 출력, KeyListener 입력 처리에 사용한다.
Java2D / BufferedImage	RenderCore, UiManager, Texture, SpriteRenderer	프레임버퍼, 픽셀 배열, UI 텍스트/이미지 합성에 사용한다.
ImageIO	Texture, UiManager, SpriteRenderer	PNG 파일을 BufferedImage로 로드한다.
Java Sound sampled API	SoundEngine	WAV BGM/SFX를 AudioInputStream과 Clip으로 재생한다.
java.nio.file	AssetRegistry, ScriptAPI	JSON/맵 파일 읽기에 사용한다.
java.io	AssetRegistryBuilder, ScriptManager, SoundEngine	파일 스캔, FileWriter, FileInputStream, 오디오 파일 접근에 사용한다.
java.util collections	대부분의 런타임 클래스	Map/List/ArrayList/HashMap/Comparator 등 런타임 캐시와 목록 관리에 사용한다.
java.util.concurrent.AtomicInteger	Entity	entityId 자동 증가에 사용한다.
Lua 표준 함수	Data/Script/*.lua	dofile/loadfile/assert, math, table, io, os, string 기능으로 콘텐츠 로직을 구성한다.

11. Lua engine API 참조

API	설명
initScene(sceneName?, mapId)	AssetRegistry에서 map.dat 경로를 찾아 int[][] 맵을 만들고 기본 player를 생성한 뒤 Boot의 활성 씬으로 설정한다.
assignWallTexture(tileId?, textureId)	맵 타일 숫자와 벽 텍스처 assetId를 연결한다. tileId 생략 시 1번 벽에 연결한다.

API	설명
setFloorTexture(textureId)	전역 바닥 텍스처를 설정한다.
setCeilingTexture(textureId)	전역 천장 텍스처를 설정한다.
spawnEntity(type, assetId, x, y, scriptId, scale?)	엔티티를 생성하고 scriptPath를 연결한 뒤 현재 씬에 추가한다. entityId를 반환한다.
getEntity(entityId)	현재 씬에서 entityId로 엔티티를 찾아 Lua userdata로 반환한다.
setEntityScale(entityId, scale)	엔티티 RenderComponent.scale을 변경한다.
attachUi(textureId, visible?, currentTextureId?)	플레이어 엔티티에 UiComponent를 붙인다.
updateUiTexture(entityId?, textureId)	플레이어 또는 지정 엔티티의 UI currentTextureId를 갱신한다.
uiClear()	프레임 UI draw command를 모두 제거한다.
uiRect(x, y, w, h, color, alpha?)	사각형 UI draw command를 추가한다.
uiImage(textureId, x, y, w?, h?, alpha?)	UI 이미지를 원본 크기 또는 지정 크기로 그린다.
uiImageRotated(textureId, x, y, w?, h?, angle, alpha?)	UI 이미지를 회전해 그린다.
uiText(text, x, y, size, color, alpha?)	좌상단 기준 텍스트를 그린다.
uiTextCenter(text, x, y, size, color, alpha?)	중앙 정렬 텍스트를 그린다.
setupPlayer(x, y, dirX, dirY, planeX, planeY)	플레이어 위치와 카메라 방향/투영 평면을 설정한다.
playSound(entityId, soundId, loop?)	엔티티 SoundComponent에 SFX 재생 요청을 기록한다.
playBgm(soundId, loop?)	AssetRegistry의 WAV 경로를 찾아 BGM을 재생한다.
stopBgm()	현재 BGM을 정지한다.
exit()	프로세스를 종료한다.
hasWallBetween(x1, y1, x2, y2)	선분 샘플링으로 시야/타격 사이 벽을 검사한다.
distance(entityA, entityB)	두 엔티티 위치 사이의 유클리드 거리를 반환한다.

12. 현재 구현상 주의점

- Config.java는 src/engine/config.json을 실제 파싱하지 않고 기본값을 사용한다.
- SceneLoader.loadMap은 경로 조회만 하고 실제 로더로 쓰이지 않는다. 현재 씬 로드는 ScriptAPI.initScene에 집중되어 있다.
- scene.json은 존재하지만 실질적인 씬 선언 데이터로 사용되지 않는다.
- PhysicsCore의 active/passive/animation 확장 메서드는 비어 있어 Java 물리 로직은 이동/벽 충돌 중심이다.
- UI Lua 스크립트는 1280x720 좌표를 하드코딩한다. 해상도 변경 시 Lua UI 수정이 필요하다.
- AssetRegistryBuilder는 0x00~0xFF 8비트 index 제한을 가진다.
- 같은 scriptPath를 사용하는 엔티티는 ScriptManager의 동일 Lua env/update cache를 공유한다. per-entity 상태는 _G.monster_states 같은 별도 테이블로 관리해야 한다.
- Lua에서 entity.physics, entity.render 등 Java public 필드에 직접 접근하므로 필드명 변경은 콘텐츠 스크립트와 강하게 연결된다.

13. 콘텐츠 추가 작업 흐름

작업	절차	주의
새 이미지/오디오 추가	Data 아래 적절한 폴더에 파일 추가 후 실행 시 AssetRegistryBuilder가 Data.json에 반영한다.	파일 수가 256개 제한을 넘지 않는지 확인한다.
새 몬스터 추가	Data/Char에 이미지 추가, Data/Script/monster_x.lua 작성, object_spawn.lua SCRIPT_BY_TYPE에 매핑한다.	AI 공통 로직은 monster_common.lua를 재사용한다.
새 아이템/무기 추가	아이템 스크립트 작성 후 ObjectSpawner와 object_spawn.lua에 타입을 연결한다.	획득 상태는 _G.PlayerState에 반영하는 패턴을 따른다.
새 씬 추가	Data/Level/<scene>/map.dat와 Lua scene controller를 만들고 engine.initScene + spawnController 패턴을 사용한다.	scene.json은 현재 자동 로딩되지 않는다.
UI 추가	Textures.UI.* 에셋을 추가하고 Lua에서 engine.uimage/uiText/uiRect draw command를 호출한다.	해상도 좌표는 현재 1280x720 기준이다.

부록 A. Java 메서드 카탈로그

아래 목록은 src/engine 아래 프로젝트 정의 Java 메서드를 클래스별로 정리한 것이다. 익명 VarArgFunction.invoke 내부 구현은 Lua engine API 목록으로 별도 정리했다.

클래스	메서드	역할
engine.Core	Core()	부트스트랩 전체를 구성한다. 설정, 에셋 레지스트리, 텍스처 /UI/사운드, Lua ScriptManager, 입력, 물리, 렌더러를 생성한다.
engine.Core	initWindow()	JFrame과 Canvas 크기를 구성하고 화면을 표시한다.
engine.Core	start()	메인 루프 스레드를 시작한다.
engine.Core	run()	60Hz 업데이트 누적 델타 루프와 렌더 호출을 수행한다.
engine.Core	update(double)	입력 스냅샷, 플레이어 이동, 엔티티 Lua update, 사운드 요청 처리, 물리 업데이트를 순서대로 실행한다.
engine.Core	render()	RenderCore 결과 프레임버퍼를 BufferStrategy로 화면에 출력한다.
engine.Core	halt(String)	치명 오류를 출력하고 프로세스를 종료한다.
engine.Core	stop()	메인 루프 종료와 스레드 join을 수행한다.
engine.Core	main(String[])	애플리케이션 진입점이다.
engine.Control	getLuaWrapper()	입력 상태 객체를 Lua에서 접근 가능한 Java userdata로 변환한다.
engine.Control	keyPressed(KeyEvent)	WASD, 방향키, Enter, Space, M/O/P/U 키의 press 상태를 갱신한다.
engine.Control	keyReleased(KeyEvent)	키 release 상태를 반영한다.
engine.Control	keyTyped(KeyEvent)	현재 미사용 키 입력 콜백이다.
engine.Control	snapshot()	volatile 입력 상태를 프레임 단위 stable 상태인 s_* 필드로 복사한다.
engine.Scene	Scene(String,int[][])	씬 이름과 타일맵을 보관하는 런타임 씬을 생성한다.
engine.Scene	addEntity(Entity)	씬 엔티티 목록에 엔티티를 추가한다.
engine.Scene	getEntities()	엔티티 목록을 반환한다.
engine.Scene	setPlayer(Entity)	플레이어 엔티티를 설정하고 씬 엔티티 목록에도 등록한다.
engine.Scene	getPlayer()	현재 씬의 플레이어를 반환한다.
engine.Scene	getTile(int,int)	좌표 범위 밖은 벽으로 간주하고 타일값을 반환한다.
engine.Scene	getEntityById(int)	entityId로 엔티티를 선형 탐색한다.
engine.Scene	getMap()	내부 int[][] 맵을 반환한다.
engine.Scene	getWidth()	맵 가로 길이를 반환한다.
engine.Scene	getHeight()	맵 세로 길이를 반환한다.
engine.Scene	getName()	씬 이름을 반환한다.
engine.boot.Boot	Boot()	부트 객체 생성자다.
engine.boot.Boot	loadAssets(Texture,UiManager)	AssetRegistry를 로드하는 보조 메서드다. 현재 Core에서는 직접 호출하지 않는다.
engine.boot.Boot	loadConfig()	Config 기본값을 생성한다. 현재 config.json 파싱은 수행하지 않는다.

클래스	메서드	역할
engine.boot.Boot	init(Texture)	Textures 카테고리의 에셋 ID를 Texture에 로드시킨다.
engine.boot.Boot	getConfig()	현재 Config 객체를 반환한다.
engine.boot.Boot	getCurrentScene()	활성 씬을 반환한다.
engine.boot.Boot	setAndActivateScene(Scene)	활성 씬을 교체한다.
engine.boot.Config	Config()	해상도, 스케일, FPS, 디버그, 볼륨 기본값을 설정한다.
engine.boot.Config	getDisplayWidth()	baseWidth * scale 값을 반환한다.
engine.boot.Config	getDisplayHeight()	baseHeight * scale 값을 반환한다.
engine.boot.AssetRegistry	loadFromJson(String)	Data/Data.json을 읽어 id/path/index 맵을 구성한다.
engine.boot.AssetRegistry	registerPath(String,String)	런타임에 id와 path를 직접 등록한다.
engine.boot.AssetRegistry	getAllIds()	등록된 전체 id set을 반환한다.
engine.boot.AssetRegistry	parseCategory(JSONObject,String)	JSON 카테고리 배열을 순회해 레지스트리 맵에 등록한다.
engine.boot.AssetRegistry	getPath(String)	id 또는 index 문자열로 경로를 조회한다.
engine.boot.AssetRegistry	getPath(int)	정수 index로 경로를 조회한다.
engine.boot.AssetRegistry	getIdByIndex(String)	index 문자열을 id로 변환한다.
engine.boot.AssetRegistry	getIdByIndex(int)	정수 index를 id로 변환한다.
engine.boot.AssetRegistry	getPathByIndex(String)	index 문자열로 경로를 조회한다.
engine.boot.AssetRegistry	getIndex(String)	id 또는 index 문자열을 0~255 정수 index로 변환한다.
engine.boot.AssetRegistry	getIdsByCategory(String)	prefix로 시작하는 id 목록을 반환한다.
engine.boot.AssetRegistry	parseIndex(String)	0xNN 또는 십진 index 문자열을 검증/파싱한다.
engine.boot.AssetRegistryBuilder	AssetRegistryBuilder()	빌더 생성자다.
engine.boot.AssetRegistryBuilder	buildRegistry()	Data 폴더를 스캔하고 Data/Data.json을 재생성한다.
engine.boot.AssetRegistryBuilder	scanRecursively(File,Map,int)	디렉터리를 재귀 순회하며 런타임 에셋을 카테고리화한다.
engine.boot.AssetRegistryBuilder	isRuntimeAsset(File)	png, wav, dat, json, lua 파일만 런타임 에셋으로 인정한다.
engine.boot.AssetRegistryBuilder	buildJsonString(Map)	스캔 결과를 Data.json 문자열로 직렬화한다.
engine.boot.AssetRegistryBuilder	writeJson(String)	생성된 레지스트리 JSON을 파일로 저장한다.
engine.boot.SceneLoader	SceneLoader(Scene)	씬 로더 생성자다. 현재 기능은 제한적이다.
engine.boot.SceneLoader	loadMap(String)	AssetRegistry에서 맵 경로를 찾지만 실제 맵 파싱은 구현되어 있지 않다.
engine.Entity.Entity	Entity(String,String,double,double)	기본 컴포넌트와 위치를 가진 엔티티를 생성한다.
engine.Entity.Entity	getLuaWrapper()	엔티티를 Lua userdata로 변환해 캐싱한다.
engine.Entity.Entity	getPhysics()	PhysicsComponent를 반환한다.
engine.Entity.Entity	toString()	디버그용 엔티티 문자열을 만든다.

클래스	메서드	역할
engine.Entity.PhysicsComponent	stop()	속도를 0으로 초기화한다.
engine.Entity.PlayerController	update(Entity,Control,double)	입력 상태에 따라 플레이어 이동 속도와 카메라 회전을 갱신한다.
engine.Entity.PlayerController	rotate(CameraComponent,double)	카메라 방향 벡터와 평면 벡터를 회전시킨다.
engine.Entity.SoundComponent	SoundComponent()	사운드 요청 상태 기본값을 설정한다.
engine.Entity.SoundComponent	play(String,boolean)	엔티티 단위 사운드 재생 요청을 표시한다.
engine.Entity.SoundComponent	stop()	현재 사운드 요청을 해제한다.
engine.Entity.SoundComponent	clear()	requestPlay 플래그를 해제한다.
engine.physics.PhysicsCore	update(Scene,double)	동적 엔티티 이동, 활성화/비활성 혹은 애니메이션 tick을 순회한다.
engine.physics.PhysicsCore	applyMovement(Entity,Scene,double)	가속도/속도를 적용하고 축별 벽 충돌을 처리한다.
engine.physics.PhysicsCore	isWall(Scene,double,double)	엔티티 반경 기준으로 네 모서리 벽 충돌을 검사한다.
engine.physics.PhysicsCore	updateActiveLogic(Entity,double)	활성 엔티티용 확장 지점이다. 현재 비어 있다.
engine.physics.PhysicsCore	updatePassiveLogic(Entity,double)	비활성 엔티티용 확장 지점이다. 현재 비어 있다.
engine.physics.PhysicsCore	updateAnimationTick(Entity,double)	렌더 애니메이션 tick 확장 지점이다. 현재 실질 로직은 없다.
engine.physics.PhysicsCore	activatePhysics(Entity)	엔티티를 동적 물리 대상으로 표시한다.
engine.render.Texture	loadTextures(String[])	AssetRegistry 경로에서 텍스처 파일을 읽고 벽/flat 저장소에 등록한다.
engine.render.Texture	addWallTexture(int,int,int,int[])	맵 타일 번호와 벽 픽셀 배열을 연결한다.
engine.render.Texture	addFlatTexture(int,int,int,int[])	바닥/천장/UI 계열 flat 텍스처를 index로 저장한다.
engine.render.Texture	setGlobalFloorTexture(String)	전역 바닥 텍스처 index를 설정한다.
engine.render.Texture	setGlobalCeilingTexture(String)	전역 천장 텍스처 index를 설정한다.
engine.render.Texture	addAsset(String,int,int,int,int)	다방향/다프레임 assetLibrary 슬롯을 생성한다. 현재 주류 렌더 경로에서는 거의 쓰이지 않는다.
engine.render.Texture	setPixels(String,int,int,int[])	assetLibrary의 특정 방향/프레임 픽셀을 설정한다.
engine.render.Texture	getPixel(String,int,int,double,double)	assetLibrary에서 정규화 좌표 픽셀을 조회한다.
engine.render.Texture	getWallPixel(int,double,double)	벽 타일 ID와 UV 좌표로 벽 픽셀을 조회한다.
engine.render.Texture	calculateDirIndex(double,double)	상대 각도를 8방향 인덱스로 변환한다.
engine.render.Texture	hasMultipleDirections(String)	특정 assetId가 방향별 이미지를 가진 것으로 볼지 판단한다.
engine.render.Texture	addWallTextureWithStringId(String,int,int,int[])	문자열 assetId 기반 벽 텍스처를 asset index 저장소에 넣는다.
engine.render.Texture	bindIntIdToStringId(int,String)	맵 타일 숫자를 문자열 벽 텍스처 id에 연결한다.
engine.render.Texture	loadWallTexture(int,String)	경로 기반 벽 텍스처 로드 보조 메서드다.
engine.render.Texture	getFloorPixel(double,double)	전역 바닥 텍스처에서 반복 UV 픽셀을 조회한다.
engine.render.Texture	getCeilingPixel(double,double)	전역 천장 텍스처에서 반복 UV 픽셀을 조회한다.

클래스	메서드	역할
engine.render.Texture	getFlatPixel(int,double,double)	flat 텍스처 index와 반복 UV로 픽셀을 조회한다.
engine.render.RenderCore	RenderCore(int,int,int,Texture)	프레임버퍼, zBuffer, Shader, SpriteRenderer를 초기화한다.
engine.render.RenderCore	getFrameBuffer()	렌더링된 BufferedImage를 반환한다.
engine.render.RenderCore	render(Scene,UiManager)	클리어, 바닥/천장, 벽, 스프라이트, UI 순서로 한 프레임을 렌더한다.
engine.render.RenderCore	clear()	프레임버퍼를 검정색으로 채운다.
engine.render.RenderCore	renderUiOverlay(Scene,UiManager)	엔티티 UiComponent 기반 고정 UI 이미지를 픽셀 배열에 합성한다.
engine.render.RenderCore	renderFloorAndCeiling(Entity)	플레이어 카메라 기준으로 바닥/천장 텍스처를 투영한다.
engine.render.RenderCore	renderWalls(Scene,Entity)	DDA 레이캐스팅으로 벽 컬럼을 그리고 zBuffer를 채운다.
engine.render.SpriteRenderer	SpriteRenderer(Texture,Shader)	스프라이트 렌더러를 생성한다. Texture 인자는 현재 사용하지 않는다.
engine.render.SpriteRenderer	loadSprite(Entity)	엔티티 assetId에 대응하는 이미지를 lazy-load하고 캐싱한다.
engine.render.SpriteRenderer	render(Scene,int[],int,int,double[])	활성 스프라이트를 거리 역순으로 정렬해 렌더한다.
engine.render.SpriteRenderer	distanceSq(Entity,Entity)	정렬용 제곱 거리 값을 계산한다.
engine.render.SpriteRenderer	renderSprite(Entity,Entity,int[],int,int,double[])	월드 좌표를 카메라 공간으로 투영하고 zBuffer 뒤 픽셀은 제외해 그린다.
engine.render.Shader	applyFog(int,double)	거리 기반 안개 색상 보간을 적용한다.
engine.render.Shader	applyPosterize(int,int)	RGB 채널을 계단화한다. 현재 주요 렌더 경로에서는 사용 빈도가 낮다.
engine.render.Shader	tint(int,int,double)	원본 색과 틴트 색을 intensity로 보간한다.
engine.render.UiManager	UiManager()	UI 관리자 생성자다.
engine.render.UiManager	registerUi(String,String)	UI 이미지 파일을 로드하고 index 기반 캐시에 저장한다.
engine.render.UiManager	getPixels(String)	UI 텍스처 픽셀 배열을 반환한다.
engine.render.UiManager	getWidth(String)	UI 텍스처 폭을 반환한다.
engine.render.UiManager	getHeight(String)	UI 텍스처 높이를 반환한다.
engine.render.UiManager	clearDrawCommands()	이번 프레임 UI draw command 목록을 비운다.
engine.render.UiManager	drawRect(int,int,int,int,int,double)	사각형 draw command를 추가한다.
engine.render.UiManager	drawImage(String,int,int,int,int,int,double)	이미지 draw command를 추가한다.
engine.render.UiManager	drawRotatedImage(String,int,int,int,int,int,double,double)	회전 이미지 draw command를 추가한다.
engine.render.UiManager	drawText(String,int,int,int,int,int,double)	좌상단 기준 텍스트 draw command를 추가한다.
engine.render.UiManager	drawTextCentered(String,int,int,int,int,int,double)	중앙 정렬 텍스트 draw command를 추가한다.
engine.render.UiManager	renderDrawCommands(BufferedImage)	Graphics2D로 누적 UI 명령을 프레임버퍼에 렌더한다.
engine.render.UiManager	toColor(int)	0xRRGGBB 정수를 Color로 변환한다.
engine.audio.SoundEngine	SoundEngine()	사운드 엔진 초기화 로그를 출력한다.
engine.audio.SoundEngine	playBgm(String,boolean)	WAV 파일을 Clip으로 열어 BGM으로 재생한다.
engine.audio.SoundEngine	stopBgm()	현재 BGM Clip을 중지/닫기 처리한다.

클래스	메서드	역할
engine.audio.SoundEngine	playSfx(String)	효과음 Clip을 재생하고 STOP 이벤트에서 정리한다.
engine.audio.SoundEngine	stopAllSfx()	활성 효과음 Clip을 모두 중지/정리한다.
engine.audio.Listener	Listener(SoundEngine)	SoundEngine을 주입받는 이벤트 리스너 생성자다.
engine.audio.Listener	onCollision(String)	충돌 이벤트 사운드를 재생한다.
engine.audio.Listener	onSceneChange(String)	씬 변경 시 BGM을 반복 재생한다.
engine.script.ScriptManager	ScriptManager(Boot,Texture,UiManager,SoundEngine)	Lua globals를 만들고 ScriptAPI를 등록한다.
engine.script.ScriptManager	loadScript(String,boolean)	Lua 파일을 별도 env로 로드하고 update 함수를 캐싱한다.
engine.script.ScriptManager	updateEntity(Entity,double,Entity,Control)	엔티티 스크립트의 update(entity,dt,player,control)를 호출한다.
engine.script.ScriptManager	resetAllScripts()	스크립트 env와 update 캐시를 전부 비운다.
engine.script.ScriptManager	runScript(String)	AssetRegistry id로 Lua 스크립트를 강제 로드/실행한다.
engine.script.ScriptManager	reloadScript(String)	특정 Lua 파일의 env/cache를 제거하고 재로드한다.
engine.script.ScriptAPI	ScriptAPI(Boot,Texture,UiManager,SoundEngine)	Lua API가 접근할 엔진 서비스 참조를 저장한다.
engine.script.ScriptAPI	register(Globals)	engine 테이블과 Lua 호출 함수를 globals에 등록한다.
engine.script.ScriptAPI	hasWallBetween(double,double,double,double)	두 월드 좌표 사이를 샘플링해 벽 존재 여부를 검사한다.
engine.script.ScriptAPI	resolveAssetId(String)	index 문자열이면 asset id로 변환하고 아니면 원문 id를 반환한다.

부록 B. Lua 함수 카탈로그

아래 목록은 Data/Script 아래 Lua 파일에 정의된 local/function/module 메서드를 정리한 것이다.

스크립트	함수	역할
combat_effects.lua	hitDirection(player,target)	플레이어 기준 타격 방향 벡터를 계산한다.
combat_effects.lua	CombatEffects.markEnemyHit(player,target)	피격 플래시, 넉백, 크로스헤어 피드백을 기록한다.
combat_effects.lua	CombatEffects.tickEnemy(entity,mem,dt)	피격 플래시 타이머와 스프라이트 복구를 처리한다.
debug_hotkey.lua	DebugHotkey.consume(control)	디버그 전환 키 입력을 소비하고 디버그 룸으로 이동한다.
debug_room.lua	clamp(value,min,max)	값을 범위로 제한한다.
debug_room.lua	setNotice(text)	디버그 룸 알림 문구를 설정한다.
debug_room.lua	buildDebugMaze()	테스트용 맵 데이터를 생성한다.
debug_room.lua	setupWorld()	벽/바닥/천장 텍스처를 설정한다.
debug_room.lua	resetDebugState()	디버그 룸 상태와 플레이어 상태를 초기화한다.
debug_room.lua	spawnController()	디버그 룸 controller 엔티티를 스폰한다.
debug_room.lua	loadDebugRoom()	디버그 씬을 로드하고 플레이어를 배치한다.
debug_room.lua	goToTitle()	타이틀 스크립트를 로드한다.
debug_room.lua	spawnPoint(player)	플레이어 앞 스폰 좌표를 계산한다.
debug_room.lua	spawnSelected(player)	선택한 몬스터/아이템을 스폰한다.
debug_room.lua	findAttackTarget(player,range)	디버그 공격 대상 엔티티를 찾는다.
debug_room.lua	attack(player)	디버그 룸 공격 처리를 수행한다.
debug_room.lua	updateNotice(dt)	알림 타이머를 갱신한다.
debug_room.lua	keepPlayerAlive()	디버그 중 플레이어 사망을 막는다.
debug_room.lua	drawDebugMenu()	디버그 스폰 메뉴를 그린다.
debug_room.lua	draw(player)	디버그 HUD를 그린다.
debug_room.lua	update(entity,dt,player,control)	디버그 룸 controller 프레임 로직이다.
end.lua	setupWorld()	엔딩 씬 텍스처를 설정한다.
end.lua	spawnController()	엔딩 controller 엔티티를 스폰한다.
end.lua	loadScene()	엔딩 씬을 로드한다.
end.lua	choose()	엔딩 메뉴 선택 액션을 예약한다.
end.lua	runPendingAction()	예약된 타이틀 이동/종료 액션을 실행한다.
end.lua	draw()	엔딩 메뉴 UI를 그린다.
end.lua	update(entity,dt,player,control)	엔딩 메뉴 입력과 렌더를 처리한다.
escape.lua	setupWorld()	탈출 씬 텍스처를 설정한다.
escape.lua	spawnController()	탈출 controller 엔티티를 스폰한다.
escape.lua	loadScene()	탈출 씬을 로드한다.
escape.lua	choose()	탈출 메뉴 선택 액션을 예약한다.
escape.lua	runPendingAction()	예약된 타이틀 이동/종료 액션을 실행한다.

스크립트	함수	역할
escape.lua	draw()	탈출 메뉴 UI를 그린다.
escape.lua	update(entity,dt,player,control)	탈출 메뉴 입력과 렌더를 처리한다.
HybridMazeGenerator.lua	new(width,height,seed)	하이브리드 미로 생성기 객체를 만든다.
HybridMazeGenerator.lua	generate()	맵과 오브젝트를 한번에 생성한다.
HybridMazeGenerator.lua	generateMap()	방, DFS 미로, 루프를 포함한 맵을 만든다.
HybridMazeGenerator.lua	generateObjects()	시작점, 출구, 적, 아이템, 무기를 배치한다.
HybridMazeGenerator.lua	saveToFile(folderPath)	map.dat 형식으로 맵을 저장한다.
HybridMazeGenerator.lua	fillWithWalls()	전체 맵을 벽으로 초기화한다.
HybridMazeGenerator.lua	generateRooms(numRooms)	랜덤 방을 생성한다.
HybridMazeGenerator.lua	generateMaze()	남은 벽 영역에 DFS 미로를 생성한다.
HybridMazeGenerator.lua	runDFS(startX,startY)	스택 기반 DFS로 통로를 판다.
HybridMazeGenerator.lua	connectRoomsAndMakeLoops()	방 연결과 일부 루프를 만든다.
HybridMazeGenerator.lua	spawnObjects()	오브젝트 생성 함수의 별칭이다.
HybridMazeGenerator.lua	getDistance(x1,y1,x2,y2)	맨해튼 거리를 계산한다.
HybridMazeGenerator.lua	spawnExit()	시작점에서 먼 출구를 배치한다.
HybridMazeGenerator.lua	spawnAtRandomPath(objectType)	무작위 길 타일에 오브젝트를 배치한다.
HybridMazeGenerator.lua	addObject(objectType,x,y)	오브젝트 배열과 점유 맵에 배치 정보를 추가한다.
item_ammo.lua	ensureVisual(entity)	탄약 아이템 외형과 스케일을 보장한다.
item_ammo.lua	update(entity,dt,player,control)	플레이어 접근 시 탄약을 지급하고 비활성화한다.
item_heal.lua	ensureVisual(entity)	회복 아이템 외형과 스케일을 보장한다.
item_heal.lua	update_item_heal(entity,dt,player)	플레이어 접근 시 체력을 회복한다.
item_heal.lua	update(entity,dt,player,control)	회복 아이템 controller 진입점이다.
loading.lua	setupWorld()	로딩 씬 텍스처를 설정한다.
loading.lua	findExit(maze,start)	생성기 출구가 없을 때 가장 먼 출구 후보를 찾는다.
loading.lua	publishMazeState(objects)	생성된 미로/오브젝트를 _G.GameState에 게시한다.
loading.lua	finishGeneration()	오브젝트 배치, map.dat 저장, 상태 게시를 완료한다.
loading.lua	prepareLoading()	로딩 씬과 animated 미로 생성을 초기화한다.
loading.lua	drawMazePreview()	미로 생성 진행 상황을 UI 미니맵으로 표시한다.
loading.lua	goToMaze()	maze.lua를 로드해 플레이 씬으로 전환한다.
loading.lua	update(entity,dt,player,control)	로딩 controller 프레임 로직이다.
maze.lua	centerOf(tile)	타일 좌표를 월드 중심 좌표로 변환한다.
maze.lua	setupWorld()	미로 씬 텍스처를 설정한다.
maze.lua	spawnController()	미로 controller 엔티티를 스폰한다.
maze.lua	spawnMazeObject(object)	GameState 오브젝트 정의를 실제 엔티티로 스폰한다.
maze.lua	resetPlayerState()	플레이어 체력/무기/탄약 상태를 초기화한다.

스크립트	함수	역할
maze.lua	loadMaze()	map.dat 씬 로드, 플레이어 배치, 오브젝트 스폰을 수행한다.
maze.lua	distTo(player,point)	플레이어와 타일 포인트 간 거리를 계산한다.
maze.lua	goToEscape()	탈출 씬으로 전환한다.
maze.lua	goToEnd()	엔딩 씬으로 전환한다.
maze.lua	setNotice(text,duration)	화면 알림 메시지를 설정한다.
maze.lua	findAttackTarget(player,range)	전방/거리/시야 조건에 맞는 공격 대상을 찾는다.
maze.lua	useWeapon(player)	무기 타입에 따라 탄약, 애니메이션, 피해 적용을 처리한다.
maze.lua	update(entity,dt,player,control)	메인 플레이 controller 프레임 로직이다.
MazeGenerator.lua	new(width,height,seed)	현재 주 사용 미로 생성기 객체를 만든다.
MazeGenerator.lua	generate()	동기 방식으로 전체 미로를 생성한다.
MazeGenerator.lua	startAnimatedGeneration()	로딩 화면용 단계적 생성 상태를 초기화한다.
MazeGenerator.lua	advanceAnimationScan()	다음 DFS 시작 후보를 찾거나 생성 완료 후 후처리한다.
MazeGenerator.lua	stepAnimatedDfs()	animated DFS를 한 단계 진행한다.
MazeGenerator.lua	stepAnimatedGeneration(maxSteps)	여러 generation step을 진행하고 완료 여부를 반환한다.
MazeGenerator.lua	getAnimationProgress()	0~1 진행률을 반환한다.
MazeGenerator.lua	isAnimationFinished()	생성 완료 여부를 반환한다.
MazeGenerator.lua	getAnimationCursor()	현재 DFS 커서 위치를 반환한다.
MazeGenerator.lua	fillWithWalls()	맵 전체를 벽으로 초기화한다.
MazeGenerator.lua	generateRooms(numRooms)	작은 방을 랜덤 배치한다.
MazeGenerator.lua	canPlaceRoom(startX,startY,w,h)	방 배치 가능 여부를 검사한다.
MazeGenerator.lua	generateMazePaths()	남은 영역을 DFS 미로로 채운다.
MazeGenerator.lua	runDFS(startX,startY)	스택 기반 DFS로 통로를 판다.
MazeGenerator.lua	connectRoomsAndMakeLoops()	일부 벽을 열어 루프를 만든다.
MazeGenerator.lua	shuffleList(list)	배열을 무작위 셔플한다.
MazeGenerator.lua	rememberRoomEntrance(room,entrance)	방 출입구 중복을 방지해 기록한다.
MazeGenerator.lua	openRoomEntrance(room,entrance)	방 벽을 열고 출입구로 기록한다.
MazeGenerator.lua	addRoomEntranceCandidate(...)	방 출입구 후보를 수집한다.
MazeGenerator.lua	collectRoomEntranceCandidates(room)	방 주변 출입구 후보 목록을 만든다.
MazeGenerator.lua	ensureRoomEntrances(minEntrances)	각 방의 최소 출입구 수를 보장한다.
MazeGenerator.lua	saveToFile(folderPath)	map.dat로 맵을 저장한다.
menu_ui.lua	clamp(value,min,max)	값을 범위로 제한한다.
menu_ui.lua	textCenterShadow(text,x,y,size,color,alpha)	그림자 포함 중앙 텍스트를 그린다.
menu_ui.lua	MenuUi.drawBackground(tintColor,tintAlpha,alpha)	메뉴 배경 이미지와 틴트를 그린다.
menu_ui.lua	MenuUi.drawTitle(text,y,size,color,alpha)	메뉴 타이틀을 그린다.

스크립트	함수	역할
menu_ui.lua	MenuUi.drawButton(label,index,selected,pulseTimer,color,alpha)	메뉴 버튼을 그린다.
monster_bull.lua	initStats(entity,mem)	황소 몬스터 HP 상태를 초기화한다.
monster_bull.lua	destroyIfDead(entity,mem)	HP 0 이하일 때 비활성/파괴 처리한다.
monster_bull.lua	stop(entity)	황소 이동 속도를 0으로 만든다.
monster_bull.lua	startCharge(mem,dx,dy,distance)	돌진 방향/타이머를 설정한다.
monster_bull.lua	update(entity,dt,player,control)	황소 AI, 돌진, 공격, 스프라이트 갱신을 처리한다.
monster_common.lua	MonsterCommon.ensure(entity,mem,config)	몬스터 공통 외형/이름/스케일 초기화를 보장한다.
monster_common.lua	MonsterCommon.face(mem,dx,dy,distance)	몬스터 바라보는 방향을 갱신한다.
monster_common.lua	MonsterCommon.canDetectPlayer(...)	시야각, 거리, 벽 여부로 플레이어 감지를 판정한다.
monster_common.lua	MonsterCommon.updateSprite(entity,mem,player)	플레이어 관찰 방향에 맞는 front/back/left/right 스프라이트를 고른다.
monster_ghost.lua	initStats(entity,mem)	유령 HP 상태를 초기화한다.
monster_ghost.lua	destroyIfDead(entity,mem)	유령 사망 상태를 반영한다.
monster_ghost.lua	update(entity,dt,player,control)	유령 AI와 공격/추적을 처리한다.
monster_spider.lua	initStats(entity,mem)	거미 HP 상태를 초기화한다.
monster_spider.lua	destroyIfDead(entity,mem)	거미 사망 상태를 반영한다.
monster_spider.lua	update(entity,dt,player,control)	거미 AI와 공격/추적을 처리한다.
NPC1.lua	update(entity,dt,player)	테스트 NPC의 플레이어 거리 기반 이동을 처리한다.
ObjectSpawner.lua	new()	오브젝트 스폰 관리자 객체를 만든다.
ObjectSpawner.lua	generate(maze)	스폰 후보 수집 후 출구/적/아이템/무기를 배치한다.
ObjectSpawner.lua	startSpawn()	플레이어 시작 오브젝트를 배치한다.
ObjectSpawner.lua	findSpawnPosition()	방 또는 첫 길 타일에서 시작점을 찾는다.
ObjectSpawner.lua	collectSpawnPoints()	전투/아이템 스폰 후보 타일을 수집한다.
ObjectSpawner.lua	spawnExit()	시작점에서 가장 먼 출구를 배치한다.
ObjectSpawner.lua	shufflePoints(points)	후보 점 배열을 셔플한다.
ObjectSpawner.lua	getDistanceFromStart(x,y)	시작점과의 맨해튼 거리를 구한다.
ObjectSpawner.lua	isOpenPosition(x,y)	길 타일이며 점유되지 않았는지 검사한다.
ObjectSpawner.lua	collectRoomSpawnPoints(room)	특정 방 내부 스폰 후보를 수집한다.
ObjectSpawner.lua	spawnFromPoints(typeName,points,minDistance,startDistance)	후보 목록에서 조건에 맞는 오브젝트 하나를 배치한다.
ObjectSpawner.lua	spawnRequiredRoomContents()	방마다 최소 적/아이템을 배치한다.
ObjectSpawner.lua	spawnRandom(typeName,count,minDistance)	전투 후보에서 랜덤 적을 배치한다.
ObjectSpawner.lua	spawnRandomItem(typeName,count,minDistance)	아이템 후보에서 랜덤 아이템을 배치한다.
ObjectSpawner.lua	spawnRandomFrom(...)	조건 완화 단계를 포함한 공통 랜덤 스폰 루틴이다.
ObjectSpawner.lua	isPositionValid(x,y,minDistance)	점유와 주변 오브젝트 최소 거리를 검사한다.

스크립트	함수	역할
ObjectSpawner.lua	addObject(typeName,x,y)	오브젝트 배열과 점유 맵에 항목을 추가한다.
object_spawn.lua	ObjectSpawn.spawn(typeName,x,y)	오브젝트 타입을 스크립트 id로 매핑해 engine.spawnEntity를 호출한다.
object_spawn.lua	ObjectSpawn.isMonster(typeName)	타입이 몬스터인지 판정한다.
player_damage.lua	minHp()	디버그 무적 상태에 따른 최소 HP를 반환한다.
player_damage.lua	knockbackDirection(player,attacker)	공격자 기준 넉백 방향을 계산한다.
player_damage.lua	PlayerDamage.tick(dt)	무적/피격 플래시/크로스헤어 타이머를 갱신한다.
player_damage.lua	PlayerDamage.tryHit(player,attacker,amount)	피해, 무적 타이머, 넉백, 플래시를 적용한다.
title.lua	setupWorld()	타이틀 씬 텍스처를 설정한다.
title.lua	spawnController()	타이틀 controller 엔티티를 스폰한다.
title.lua	loadTitle()	타이틀 씬을 로드한다.
title.lua	choose()	선택된 메뉴 액션을 예약한다.
title.lua	runPendingAction()	Start/Exit 예약 액션을 실행한다.
title.lua	draw()	타이틀 메뉴를 그린다.
title.lua	update(entity,dt,player,control)	타이틀 메뉴 입력과 렌더를 처리한다.
ui.lua	clamp(value,min,max)	값을 범위로 제한한다.
ui.lua	drawHp()	HP 바를 그린다.
ui.lua	drawWeapon()	무기 슬롯과 탄약 UI를 그린다.
ui.lua	drawFpsWeapon()	1인치 무기 이미지를 그린다.
ui.lua	Ui.startWeaponAnimation(weapon)	총/근접 무기 애니메이션 타이머를 시작한다.
ui.lua	Ui.tick(dt)	무기 애니메이션 타이머를 갱신한다.
ui.lua	drawLocalMap(player)	플레이어 주변 미니맵을 그린다.
ui.lua	drawFullMap(player,zoom)	전체 미로 지도를 확대율에 맞춰 그린다.
ui.lua	updateCrosshair(player,showFullMap)	크로스헤어 UI 컴포넌트 상태를 갱신한다.
ui.lua	drawDamageFlash()	피격 화면 플래시를 그린다.
ui.lua	Ui.draw(player,showFullMap,escapePrompt,noticeText,mapZoom)	플레이 HUD 전체를 구성한다.
weapon_gun.lua	ensureVisual(entity)	총 아이템 외형과 스케일을 보장한다.
weapon_gun.lua	update(entity,dt,player,control)	플레이어 접근 시 총과 탄약을 지급한다.
weapon_melee.lua	ensureVisual(entity)	근접 무기 외형과 스케일을 보장한다.
weapon_melee.lua	update(entity,dt,player,control)	플레이어 접근 시 근접 무기를 지급한다.

부록 C. 폴더 구조 요약

경로	설명
src/engine	Java 엔진 런타임 코드. Core, Scene, 입력, 물리, 렌더, 오디오, 스크립트 브리지 포함.
src/engine/Entity	Entity와 Physics/Render/Camera/Sound/UI 컴포넌트 데이터 컨테이너.

경로	설명
src/engine/boot	Config, Boot, AssetRegistry, AssetRegistryBuilder, 미완성 SceneLoader.
src/engine/render	Texture, RenderCore, SpriteRenderer, Shader, UiManager.
src/engine/script	Lua 기반 ScriptManager와 Lua 노출 API.
src/engine/audio	SoundEngine과 이벤트 Listener.
Data/Script	게임 로직, 씬 controller, 미로 생성, 몬스터/아이템/무기/HUD Lua 스크립트.
Data/Level	title/loading/test/maze/escape/end 레벨 폴더. map.dat와 scene.json 보관.
Data/Textures	벽, 바닥, 천장, UI, 오브젝트 텍스처.
Data/Char	몬스터/NPC 스프라이트.
Data/Audio	WAV 오디오 파일.
lib	Lua와 org.json 외부 JAR.